



in collaboration with



Title:

Lab 4 Task

Course:

ST5003CMD-Operating Systems, Security and Network

Institution:

Softwarica College of IT and E-Commerce

Submitted By:

Sandip Rawal (240684)

Submitted To

Mr.Rupak Rajbanshi

Submission Date:

September 9 , 2026

Table of Contents:

I. Introduction.....	4
II. Primary Data Types and OS Allocation.....	5
A. Data Types Sizes Across Architecture.....	5
B. Practical Example: Observing Data Type Sizes.....	5
III. OS Memory Segments.....	6
A. Memory Segment Overview.....	6
B. Memory Layout Visualization.....	6
Key Observation:.....	6
IV. Practical Code Examples.....	7
A. Observing Variable Addresses.....	7
B. Pointer and Memory Allocation.....	8
V. Laboratory Exercises.....	9
A. Lab 1: Data Type Size Exploration.....	9
Objective.....	9
Tasks.....	9
B. Lab 2: Memory Segment Address Mapping.....	10
Objective.....	10
Tasks.....	11
VI. Summary and Key Takeaways.....	12
VII. Knowledge Check Questions.....	13
VIII. Conclusion.....	14

List Of Figures:

Figure 1: Table showing datatype sizes.....	5
Figure 2: C code to observe datatypes.....	5
Figure 3: Compiled output shown.....	5
Figure 4: Table showing different segments.....	6
Figure 5: Relative position of different memory segments.....	6
Figure 6 : C Code to observe variable address.....	7
Figure 7: Compiled output shown.....	7
Figure 8: C code demonstrating pointer memory.....	8
Figure 9: Compiled and Address shown.....	8
Figure 10: C code demonstrating task1.....	9
Figure 11: Datatypes sizes featuring task2.....	9
Figure 12: Comparison of my output with table 1.....	10
Figure 13: Program declaring variables.....	11
Figure 14: Address printed.....	11

I. Introduction

Data types and memory management are important concepts in understanding how programs interact with the operating system. A data type determines the amount of memory required to store data and how that data should be interpreted. The size of data types can vary across different system architectures, which can affect memory allocation and address space. This lab explores these concepts using C programming and practical experiments on an Ubuntu system.

The `sizeof()` operator is used to observe the memory size of different data types, including `char`, `int`, `float`, `double`, `long`, `unsigned int`, and `long long`. The lab also examines the main memory segments of a process: Text, Data, BSS, Heap, and Stack. By printing variable addresses using `%p`, the practical exercises demonstrate where global, local, static, and dynamically allocated variables are located in memory.

These activities provide a practical understanding of how the operating system organizes, allocates, and manages memory.

II. Primary Data Types and OS Allocation

This section explains how different data types require different amounts of memory and how their sizes can vary depending on the system architecture.

A. Data Types Sizes Across Architecture

The size of a data type is related to the system architecture. The table below compares the memory size of common data types on 16-bit, 32-bit, and 64-bit systems and shows their effect on OS memory allocation.

Data Type	16-bit	32-bit	64-bit	OS Impact
char	1 byte	1 byte	1 byte	Consistent across architectures
short	2 bytes	2 bytes	2 bytes	Memory-efficient counters
int	2 bytes	4 bytes	4 bytes	Affects pointer arithmetic
long	4 bytes	4 bytes	8 bytes	Critical for 64-bit addressing
long long	8 bytes	8 bytes	8 bytes	Large file offsets
float	4 bytes	4 bytes	4 bytes	IEEE 754 standard
double	8 bytes	8 bytes	8 bytes	Precision calculations
pointer	2 bytes	4 bytes	8 bytes	Address space limit

Figure 1: Table showing datatype sizes

B. Practical Example: Observing Data Type Sizes

This example uses the C `sizeof()` operator to determine how many bytes different data types occupy in the current system.

```
sandip-rawal@sandip-rawal-VirtualBox:~/Lab4Task$ nano datatype_sizes.c
```

```
GNU nano 8.7.1
#include <stdio.h>

int main() {
    printf("=== Data Type Sizes on This System ===\n\n");

    printf("char: %zu bytes\n", sizeof(char));
    printf("int: %zu bytes\n", sizeof(int));
    printf("float: %zu bytes\n", sizeof(float));
    printf("double: %zu bytes\n", sizeof(double));

    printf("long: %zu bytes\n", sizeof(long));
    printf("pointer: %zu bytes\n", sizeof(int*));

    return 0;
}
```

Figure 2: C code to observe datatypes

```
sandip-rawal@sandip-rawal-VirtualBox:~/Lab4Task$ gcc datatype_sizes.c -o datatype_sizes
sandip-rawal@sandip-rawal-VirtualBox:~/Lab4Task$ ./datatype_sizes
=== Data Type Sizes on This System ===

char: 1 bytes
int: 4 bytes
float: 4 bytes
double: 8 bytes
long: 8 bytes
pointer: 8 bytes
```

Figure 3: Compiled output shown

III. OS Memory Segments

When a program runs, the operating system provides it with a process address space. This address space is divided into different memory segments, with each segment serving a specific purpose.

A. Memory Segment Overview

Each memory segment stores a particular type of program data. The main segments discussed in this part are Text, Data, BSS, Heap, and Stack.

Segment	Purpose	Grows	Managed By
Text	Machine code (instructions)	Fixed	Compiler/Loader
Data	Initialized global/static variables	Fixed	Compiler/OS
BSS	Uninitialized global/static variables	Fixed	OS
Heap	Dynamically allocated memory	Upward	Programmer + OS
Stack	Local variables, function calls	Downward	Compiler + OS

Figure 4: Table showing different segments

B. Memory Layout Visualization

The process memory layout shows the relative positions of the different memory segments. The lab observes that stack addresses are high, heap addresses are in the middle range, and Data/BSS addresses are lower.

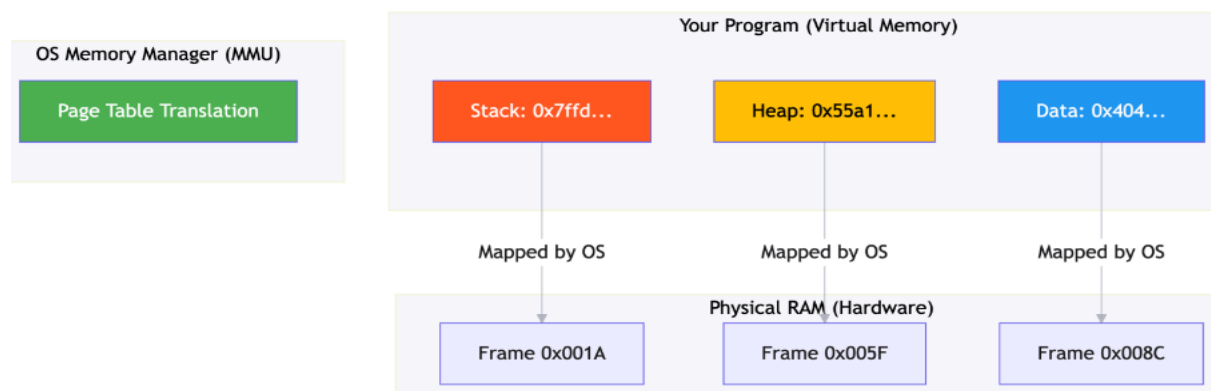


Figure 5: Relative position of different memory segments

Key Observation:

The lab notes that Stack addresses are typically higher in the **0x7ffd...** range, Heap addresses are mid range around **0x55a1...**, and Data/BSS addresses around the lower **0x404...** range.

IV. Practical Code Examples

This section demonstrates memory allocation practically using C programs. The first example observes the addresses of variables, while the second demonstrates the relationship between pointers and dynamically allocated heap memory.

A. Observing Variable Addresses

This program creates variables associated with different memory segments and prints their addresses using `%p`. This allows the memory locations of Data, BSS, Stack, and Heap variables to be observed.

```
sandip-rawal@sandip-rawal-VirtualBox:~/Lab4Task$ nano memory_segments.c

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>

int global_var = 150; // Stack
int bss_var; // BSS/Data segment

int main() {
    int local_var = 30; // Stack
    static int local_static = 20; // Data Segment

    int *heap_var = (int *)malloc(sizeof(int)); // Heap
    *heap_var = 500;

    printf(".....Variable Addresses.....\n\n");
    printf("global_var: %p (Data/BSS)\n", (void*)&global_var);
    printf("bss_var: %p (Data/BSS)\n", (void*)&bss_var);
    printf("local_var: %p (Stack)\n", (void*)&local_var);
    printf("local_static: %p (Data/BSS)\n", (void*)&local_static);
    printf("heap_var ptr: %p (Stack - pointer)\n", (void*)&heap_var);
    printf("*heap_var: %p (Heap - data)\n", (void*)heap_var);

    free(heap_var);
    return 0;
}
```

Figure 6 : C Code to observe variable address

```
sandip-rawal@sandip-rawal-VirtualBox:~/Lab4Task$ gcc memory_segments.c -o memory_segments
sandip-rawal@sandip-rawal-VirtualBox:~/Lab4Task$ ./memory_segments
.....Variable Addresses.....

global_var: 0x590010271010 (Data/BSS)
bss_var: 0x59001027101c (Data/BSS)
local_var: 0x7ffd57a1266c (Stack)
local_static: 0x590010271014 (Data/BSS)
heap_var ptr: 0x7ffd57a12670 (Stack - pointer)
*heap_var: 0x59003551b010 (Heap - data)
```

Figure 7: Compiled output shown

B. Pointer and Memory Allocation

This part demonstrates how a **pointer variable can live on the Stack while pointing to memory allocated on the Heap**. The program allocates memory using `malloc()`, stores a value in that memory, and then prints the pointer address, heap address, and stored value.

```
sandip-rawal@sandip-rawal-VirtualBox:~/Lab4Task$ nano pointer_memory.c

GNU nano 8.7.1 pointer_memory.c
#include <stdio.h>
#include <stdlib.h>

int main () {
    int *forheap; // Declare pointer (lives on STACK)
    forheap = (int *)malloc(sizeof(int)); // Allocate memory on HEAP first!
    *forheap = 30; // store value to allocated memory

    //print addresses
    printf("Address of pointer (STACK) : %p\n", (void*)&forheap);
    printf("Address of data (HEAP) : %p\n", (void*)forheap);
    printf("Value stored: %d\n", *forheap);

    return 0;
}
```

Figure 8: C code demonstrating pointer memory

```
sandip-rawal@sandip-rawal-VirtualBox:~/Lab4Task$ gcc pointer_memory.c -o pointer_memory
sandip-rawal@sandip-rawal-VirtualBox:~/Lab4Task$ ./pointer_memory
Address of pointer (STACK) : 0x7ffc850fb070
Address of data (HEAP) : 0x5679276c1010
Value stored: 30
```

Figure 9: Compiled and Address shown

V. Laboratory Exercises

A. Lab 1: Data Type Size Exploration

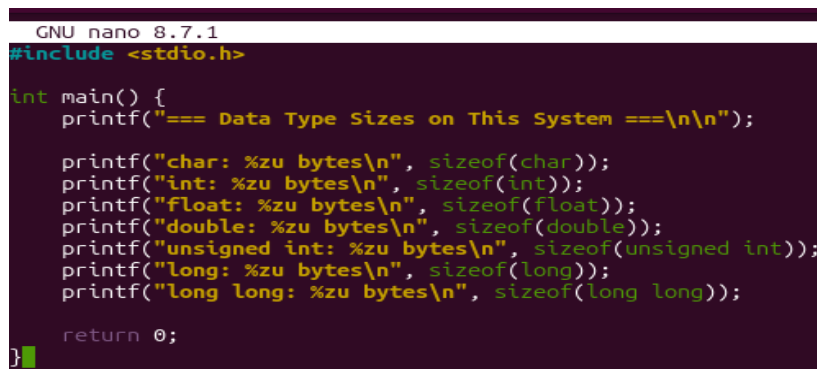
Lab 1 asks us to use `sizeof()` to check how much memory different C data types use on your Ubuntu system.

Objective

Using `sizeof()` to observe how much memory each datatype uses on your system.

Tasks

1. Create a C program that prints the size of: char, int, float, double, long, unsigned int, long long



```
GNU nano 8.7.1
#include <stdio.h>

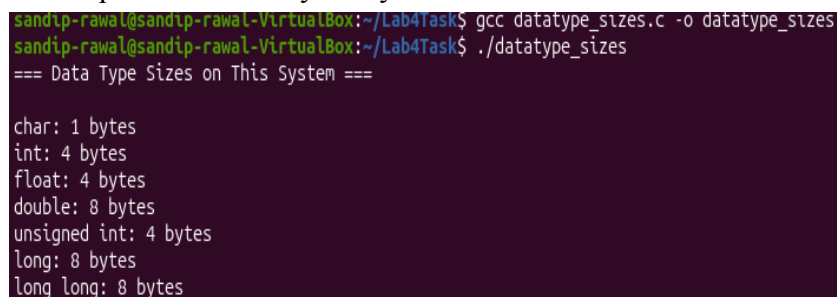
int main() {
    printf("=== Data Type Sizes on This System ===\n\n");

    printf("char: %zu bytes\n", sizeof(char));
    printf("int: %zu bytes\n", sizeof(int));
    printf("float: %zu bytes\n", sizeof(float));
    printf("double: %zu bytes\n", sizeof(double));
    printf("unsigned int: %zu bytes\n", sizeof(unsigned int));
    printf("long: %zu bytes\n", sizeof(long));
    printf("long long: %zu bytes\n", sizeof(long long));

    return 0;
}
```

Figure 10: C code demonstrating task1

2. Compile and run on your system



```
sandip-rawal@sandip-rawal-VirtualBox:~/Lab4Task$ gcc datatype_sizes.c -o datatype_sizes
sandip-rawal@sandip-rawal-VirtualBox:~/Lab4Task$ ./datatype_sizes
=== Data Type Sizes on This System ===

char: 1 bytes
int: 4 bytes
float: 4 bytes
double: 8 bytes
unsigned int: 4 bytes
long: 8 bytes
long long: 8 bytes
```

Figure 11: Datatypes sizes featuring task2

3. Compare output with Table 1

Data Type	16-bit	32-bit	64-bit
char	1 byte	1 byte	1 byte
short	2 bytes	2 bytes	2 bytes
int	2 bytes	4 bytes	4 bytes
long	4 bytes	4 bytes	8 bytes
long long	8 bytes	8 bytes	8 bytes
float	4 bytes	4 bytes	4 bytes
double	8 bytes	8 bytes	8 bytes
pointer	2 bytes	4 bytes	8 bytes

```

=== Data Type Sizes on This System ===

char: 1 bytes
int: 4 bytes
float: 4 bytes
double: 8 bytes
unsigned int: 4 bytes
long: 8 bytes
long long: 8 bytes

```

Figure 12: Comparison of my output with table 1

The output of the program matches the **64-bit values in Table I** for **char**, **int**, **float**, **double**, **long**, and **long long**. The **unsigned int** size is 4 bytes in the output, but it is not separately listed in Table I. Therefore, the results confirm that the program is running on a 64-bit system.

4. Why might the size of(long) differ between 32-bit and 64_bit systems?

- **sizeof(long)** may differ because different system architectures use different data models. In the given table is 4 bytes on a 32-bit system and 8 bytes on a 64-bit system.

B. Lab 2: Memory Segment Address Mapping

Created variables belonging to different memory segments, printed their addresses using **%p**, identified the highest and lowest addresses, and calculated the difference between Stack and Heap addresses.

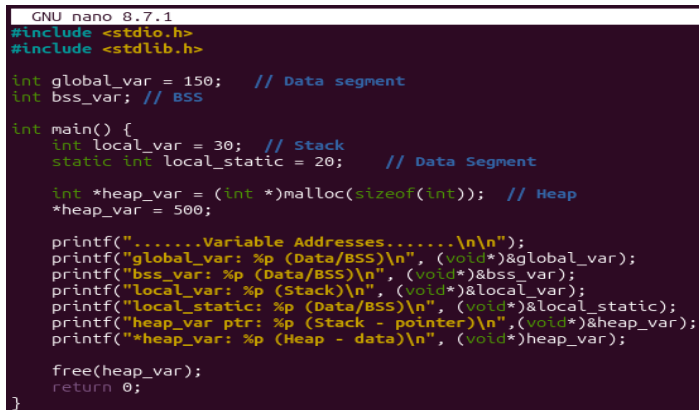
Objective

Observe where different variables live in memory and validate the OS memory layout.

Tasks

1) Write a program that declares variables in each segment:

- Global initialized variable (Data segment)
- Global uninitialized variable (BSS segment)
- Local variable (Stack segment)
- Dynamically allocated variable (Heap segment)



```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>

int global_var = 150; // Data segment
int bss_var; // BSS

int main() {
    int local_var = 30; // Stack
    static int local_static = 20; // Data Segment

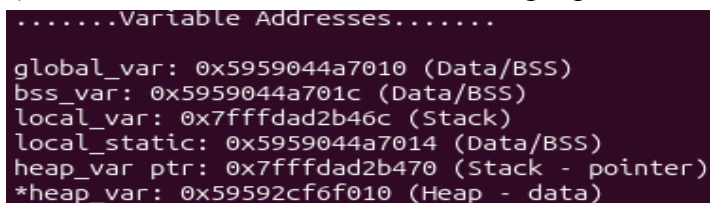
    int *heap_var = (int *)malloc(sizeof(int)); // Heap
    *heap_var = 500;

    printf(".....Variable Addresses.....\n\n");
    printf("global_var: %p (Data/BSS)\n", (void*)&global_var);
    printf("bss_var: %p (Data/BSS)\n", (void*)&bss_var);
    printf("local_var: %p (Stack)\n", (void*)&local_var);
    printf("local_static: %p (Data/BSS)\n", (void*)&local_static);
    printf("heap_var ptr: %p (Stack - pointer)\n", (void*)&heap_var);
    printf("heap_var: %p (Heap - data)\n", (void*)heap_var);

    free(heap_var);
    return 0;
}
```

Figure 13: Program declaring variables

2) Print the address of each variable using %p



```
.....Variable Addresses.....

global_var: 0x5959044a7010 (Data/BSS)
bss_var: 0x5959044a701c (Data/BSS)
local_var: 0x7fffdad2b46c (Stack)
local_static: 0x5959044a7014 (Data/BSS)
heap_var ptr: 0x7fffdad2b470 (Stack - pointer)
heap_var: 0x59592cf6f010 (Heap - data)
```

Figure 14: Address printed

3) Identify which segment has the highest addresses?

-The Stack segment has the highest addresses among the observed memory segments.

4) Identify which segment has the lowest addresses?

-The Heap segment has the lowest address among the observed segments.

5) Calculate the address difference between stack and heap?

-The output shows that the **Stack has the highest address**, while the **Heap has the lowest address among the observed segments** in this execution. The Stack address was **0x7ffdad2b46c** and the Heap address was **0x5592cf6f010**, giving an address difference of **0x2a6addbc45c**

VI. Summary and Key Takeaways

This section summarizes the main concepts learned from the lab about **data types, memory segments, OS memory management, virtual memory, and pointers**. The points below follow the wording and structure of the lab question.

Data types are contracts: They tell the OS how much memory to allocate and how to interpret the stored data.

Memory is segmented: The **Stack** stores local variables and function calls, the **Heap** stores dynamically allocated memory, **Data/BSS** stores global and static variables, and the **Text** segment stores program instructions.

The OS manages memory: The operating system tracks, allocates, protects, and virtualizes memory for processes.

Stack is automatic; Heap is manual: Stack memory is automatically managed, while Heap memory is dynamically allocated and must be managed by the programmer.

Virtual memory provides isolation: Each process has its own virtual address space, helping keep the memory of different processes separate.

Pointers bridge Stack and Heap: A pointer variable can live on the Stack while pointing to dynamically allocated memory located on the Heap.

VII. Knowledge Check Questions

1) What is a data type in the context of programming and operating systems?

-A data type specifies how much memory to allocate and how the stored data should be interpreted.

2) Name the five main memory segments in a process memory layout?

-The five segments are **Text, Data, BSS, Heap, and Stack**.

3) What is the difference between the Data segment and the BSS segment?

-The **Data segment** stores initialized global/static variables, while **BSS** stores uninitialized global/static variables.

4) Why does the OS place the stack at high virtual addresses and the text segment at low addresses?

-This arrangement organizes the process's virtual address space and allows different memory regions to be managed separately.

5) What happens to stack memory when a function returns? Who reclaims it?

-The function's stack memory is released automatically when it returns, and it is managed by the compiler/OS.

6) How does the OS know which memory belongs to which process?

-The OS uses each process's separate **virtual address space** to track and isolate its memory.

VIII. Conclusion

This lab provided a practical understanding of data types and operating system memory management using C programming. The first laboratory exercise demonstrated how different data types occupy different amounts of memory using the `sizeof()` operator. The obtained results were compared with the 64-bit values in Table I, showing the expected sizes for the tested data types.

The second laboratory exercise demonstrated how variables are placed in different memory segments, including Data, BSS, Stack, and Heap. By printing their memory addresses, it was possible to observe the differences between these segments and calculate the address difference between Stack and Heap.

The lab also demonstrated that Stack memory is automatically managed, while Heap memory is dynamically allocated. Additionally, pointers were shown to connect Stack variables with dynamically allocated Heap memory.

Overall, the practical activities helped demonstrate how data types determine memory requirements and how the operating system organizes and manages a process's virtual memory space.